

Gig-Queue: Project Report

Luca Corradini
Student ID: 83412A

August 25, 2026

1 Introduction

The project consists of a distributed, ticket pre-sale system built on **Apache Kafka**. The main goal is to simulate a *flash sale* scenario: thousands of users try to buy a limited number of seats within a few seconds. The system must serve them in a fair, ordered and verifiable way, without ever selling the same seat twice. Furthermore, the application is designed to guarantee the following non-functional properties: high availability, fault tolerance, encryption and authentication of communications. The architecture is composed of independent microservices, containerized with **Docker** and orchestrated via **Docker Compose**.

Why Kafka: The main idea behind adopting Kafka is to use it not merely as a transport layer, but as an ordered and persistent registry of the purchase queue itself. In a ticket-selling scenario, maintaining a strict *first-come, first-served* (FCFS) policy is critical. Kafka guarantees ordering within a partition; since `event_id` is used as the message key, every request for the same event is appended to the same partition, and its offset therefore records the exact arrival order for that event. Durability is provided independently by replication (Section 3.2), so no acknowledged request is lost even when a consumer fails and restarts.

2 System Architecture

The system is currently composed of 13 containers (Figure 1): three Kafka brokers, **Redis**, **AKHQ**, **kafka-init**, **mailtip**, and six application microservices. All the application services are written in **Python**: the HTTP layer uses **FastAPI**, and the Kafka clients use **confluent-kafka-python**.

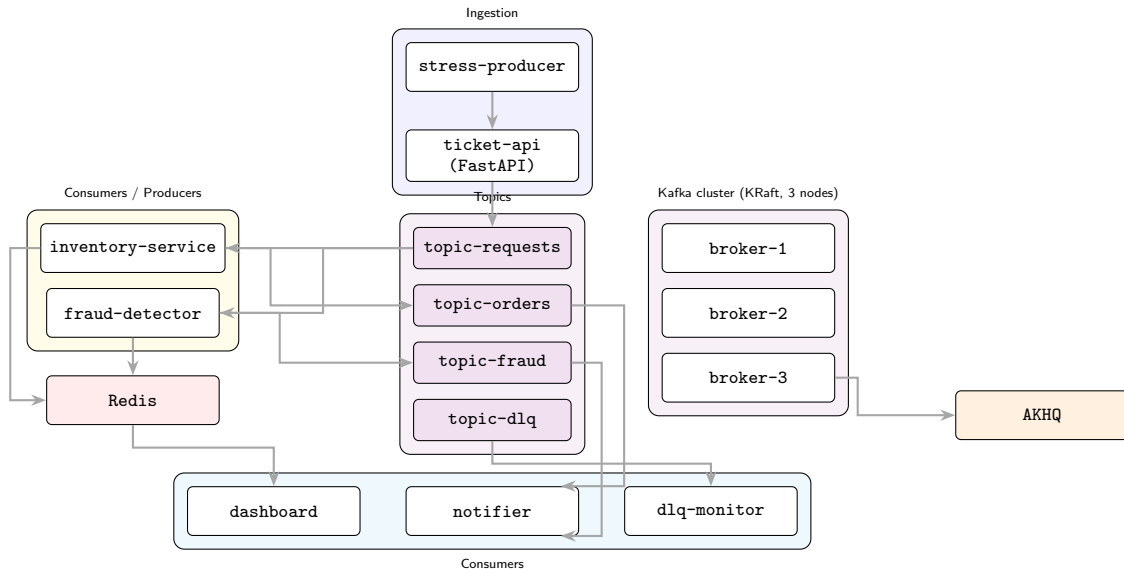


Figure 1: System architecture

2.1 Producer

The system provides two entry points that produce data into the system:

- **Ticket-api**: This is the edge layer of the system, exposing an HTTP interface. It acts as a pure Producer. Upon receiving a valid JSON payload, it produces a purchase request message to **topic-requests** and immediately returns an HTTP response to the client (POST /buy). The user can obtain the status of his order using the GET /status end-point. It relies on Redis to temporarily store the pending state of the order.
- **Stress-producer**: The service will inject a massive amount of synthetic traffic into the system, by stressing the HTTP API with high-concurrency requests to simulate specific parametric scenarios. These scenarios will include: a flash sale, a bot attack, and a fault-tolerance test.

2.2 Kafka Broker Layer

The backbone of the system is a 3-node Apache Kafka cluster running in KRaft mode, which manages metadata internally without relying on ZooKeeper by using a quorum mechanism. Automatic topic creation is disabled to ensure each topic is created with its intended partition count and configuration, and to prevent mistyped topic names from silently creating new topics. For this reason, the **kafka-init** service creates the topics with the chosen configuration, which can be seen in Table 1.

Topic	Part.	RF	min.ISR	Key	Rationale
topic-requests	3	3	2	event_id	Total ordering <i>per event</i> (queue fairness); parallelism across different events
topic-orders	1	3	2	order_id	Low frequency; confirmed and rejected outcomes distinguished by a status field
topic-fraud	1	3	2	user_id	Separate alert stream with a dedicated consumer
topic-dlq	1	3	2	—	Dead letter queue for payloads that cannot be deserialized

Table 1: Topics

2.3 Consumer-Producer Layer

These microservices act as both consumers and producers at the same time: they consume the raw requests, process them using Redis as the state store, and produce new events downstream.

- **inventory-service**: Subscribed to **topic-requests** using the **group-inventory** consumer group. It applies atomic logic to allocate seats via Redis. The decrement is executed through an idempotent Lua script (to prevent double-decrements). Based on the outcome, it acts as a producer by forwarding the result (confirmed or rejected) to **topic-orders**, or routing malformed payloads to **topic-dlq**.
- **fraud-detector**: Operates in parallel with the inventory by consuming **topic-requests** under a distinct consumer group (**group-fraud**). It maintains window counters (the key expires 60s after its first increment) on Redis to track IP and user request rates. If an anomaly is detected, it acts as a producer, publishing an alert to **topic-fraud**. To simplify this service, the fraud thresholds are static and defined a priori.

2.4 Consumer Layer

Services belonging to this category act as simple consumers: they subscribe to one or multiple topics to passively consume messages.

- **notifier**: This service is subscribed to both **topic-orders** and **topic-fraud**. It reads the final outcomes and acts as an SMTP client, forwarding emails to a local mailpit container (see Figure 2). Specifically, this consumer sends both order outcome emails to the users and security alerts about blocked users to the system administrator.
- **dlq-monitor**: Subscribed to **topic-dlq** (Dead Letter Queue). It monitors and logs discarded or malformed messages that originate from the main **topic-requests**.
- **dashboard**: The dashboard will act as a pure observer of the system. Its goal will be to provide a live overview of the system from an administrative point of view. It will not be a Kafka consumer in

the strict sense of the term, because it will not subscribe to a specific topic, but rather will rely on information fetched from the Kafka Admin API and Redis. This approach will avoid recalculating the metrics by leveraging the pre-aggregated state.

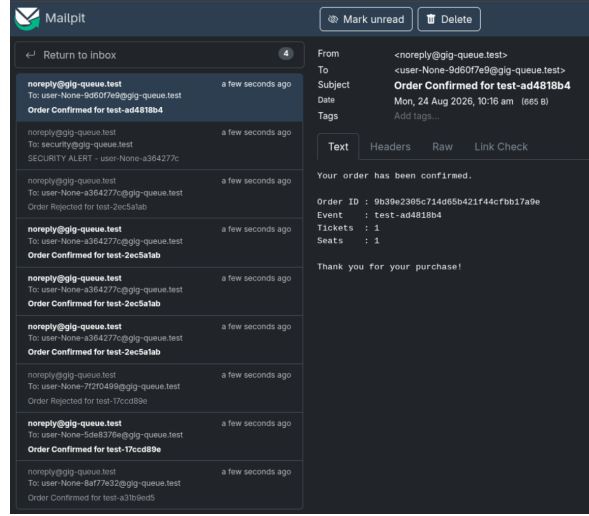


Figure 2: Mailpit interface

2.5 Redis

Redis was used as the central in-memory state store for the entire pipeline. Since the microservices communicate asynchronously through Kafka, a fast shared data-store is required to maintain the global state of the application. This allows the stateless `ticket-api` to updated users with an immediate view of their order status without having to query the Kafka brokers directly. Blocked-user flags are also stored in Redis. Table 2 summarizes the main categories of state managed by Redis.

Category	Key pattern	Written by	Read by
Queue position	<code>queue.seq:{event_id}</code> <code>queue:{order_id}</code>	Ticket API	Ticket API
Order outcome	<code>order:{order_id}</code> <code>processed:{order_id}</code> <code>queue.done:{event_id}</code>	Inventory Service	Ticket API
Seat inventory	<code>seats:{event_id}</code> <code>total:{event_id}</code>	Inventory Service	Inventory Service
Fraud counters	<code>fraud:user:*</code> <code>fraud:ip:*</code>	Fraud Detector	Fraud Detector
Block flags	<code>blocked:user:*</code> <code>blocked:ip:*</code>	Fraud Detector	Ticket API, Inventory Service

Table 2: Redis data structures.

Ticket Purchase Flow: The most critical interaction with Redis concerns ticket purchasing. Specifically, event state management is handled by the `inventory-service`. Figure 3 shows the interaction between the API, Kafka, and Redis during this operation. When a user requests a ticket, the `ticket-api` produces the request to Kafka and waits for the ack; only then does it assign a sequential queue number with an atomic increment in Redis and store the pending state. This operation order aims to prevent a misalignment of the virtual queue in the case of a producer failure. The `inventory-service` consumes the message and executes the seat allocation logic. Finally, the outcome is saved back to Redis, making it immediately available for the user.

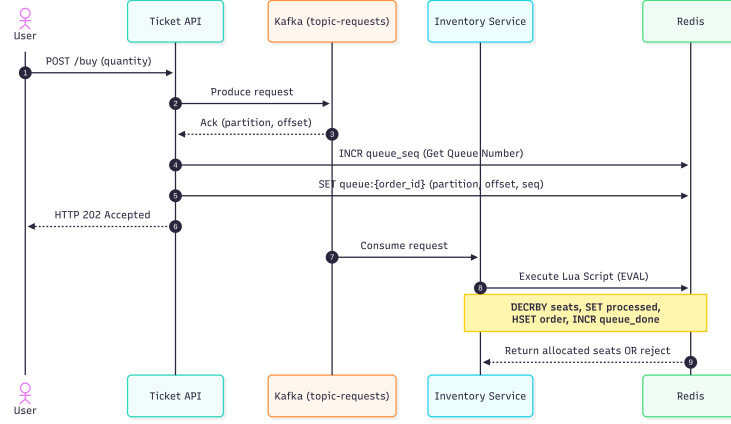


Figure 3: Ticket purchase flow

The interaction with **Redis**, especially for the **inventory-service** and the **fraud-detector**, is entirely encapsulated within Lua scripts executed via the **EVAL** command. This choice leads to two main advantages:

- **Atomicity:** **Redis** executes a Lua script as a single, uninterruptible command. No other client can observe or modify the keys between the individual reads and writes inside the script. This eliminates race conditions (e.g., overselling tickets).
- **Reduced RTT:** Since the operations involving the state saved in **Redis** are grouped into a single script, they require only one network round-trip between the microservice and the **Redis** server, significantly reducing latency.

3 Engineering Choices

3.1 Virtual Queue Number

To provide users with a clear indication of their position in the purchasing queue, the system implements a virtual queue number. Since **event_id** is used as the key for **topic-requests** (Table 1), a single Kafka partition can contain orders belonging to different events. Therefore, exposing the raw Kafka offset would carry no meaning for the user. Instead, the correct quantity is the number of orders ahead in the queue for that specific event. This is calculated dynamically using **Redis** counters:

$$\text{ahead} = \text{seq} - \text{queue_done} - 1$$

where **seq** is an atomic counter incremented by the **ticket-api** upon request ingestion, and **queue_done** is incremented by the **inventory-service** after processing each order (regardless of the final outcome). Additionally, the API estimates the remaining waiting time (**eta_seconds**), derived from the observed processing throughput of the respective Kafka partition.

3.2 Non-functional requirements

To ensure that the system survives infrastructure node failures, the **KRaft** cluster and the producers are configured as follows:

- **RF = 3:** every partition is mirrored across the three brokers, so the cluster sustains the loss of a node without losing acknowledged records.
- **min.ISR = 2:** Kafka refuses writes if fewer than two synchronized replicas are available.
- **acks=all:** the producer blocks until every in-sync replica confirms. Combined with the previous point, an acknowledged record exists on at least two independent replicas.
- **Idempotent producer:** the broker deduplicates retries, so a network timeout never creates a duplicate purchase request.

3.3 Security: Encryption, Authentication and Authorization

To secure the system, the `mTLS` protocol was chosen over `SASL` alternatives as it seamlessly covers both encryption and authentication without relying on hardcoded credentials, rejecting unauthorized clients directly during the TLS handshake. Three distinct layers secure the cluster: the first two are implemented, the third is planned.

- **Encryption:** TLS is applied to every Kafka listener, including the internal controller traffic, so data crossing the Docker network is unreadable to anyone who does not hold a valid key.
- **Authentication:** By enforcing `ssl.client.auth=required` and relying on a private **Certificate Authority** (CA), the broker is able to cryptographically verify the exact identity of each connecting microservice.
- **Authorization (ACLs):** Authentication alone will not be sufficient. A **Access Control List** (ACL) matrix will be configured, the main goal is to restrict what each authenticated client is allowed to do. For example, the `dashboard` container will act as a pure observer and will be granted *read* permissions only. In this case, if the dashboard were to be compromised by an attacker, it would still be impossible for him to inject fraudulent orders.

4 Demo and Testing

Testing : To ensure the correctness of the logic behind each microservice a suite of tests has been created. Their aim is to verify the internal behavior of the services and their interactions with the data structures.

Demos : To demonstrate the system's non-functional properties and its resilience under stress, a suite of executable scripts will be provided. The planned demonstration scenarios include:

- **Flash Sale Simulation:** A high-throughput scenario injecting massive concurrent purchase requests to demonstrate the queue fairness, the correct ordering, and the parallel processing across different events.
- **Broker Failure Resilience:** Sustained traffic injected while a Kafka broker is intentionally shut down, empirically verifying the High Availability guarantees, the partition leader re-election, and the zero data loss property.
- **Security Rejection:** Connection attempts using missing or unauthorized TLS certificates, demonstrating the effectiveness of the `mTLS` and ACL configurations in rejecting malicious traffic before it reaches the application layer.

5 Limitations and Future Work

The proposed architecture is a simplified model of a real-world scenario. The following limitations define the future improvement:

- **Simplified scenario:** The system currently models seat availability as a single numeric counter, assigning seats sequentially. So, it is not possible to choose a specific seat.
- **Single-Partition bottleneck:** Since a partition can contain orders from different events this may affect large event with massive traffic. This type of events are handled sequentially by a single consumer instance, and its load cannot be load-balanced across other available partitions. As future work, a custom partitioning strategy or dedicated partitions for high-demand events could be implemented.
- **Redis as a Single Point of Failure (SPOF):** The application state relies on a single Redis instance. A fatal crash of the Redis node affects the entire pipeline.
- **Relaxed semantics on the outcome topic:** As described in Section 2.5, there is no risk of double-selling due to the use of a Lua script in the `inventory-service`. However, because Kafka transactions are not implemented, the action of publishing the result to `topic-orders` and the action of committing the offset are not atomic. If the service crashes exactly between these two operations, the same outcome message might be published twice to the topic. Downstream consumers (like the `notifier`) easily tolerate this by deduplicating the messages using the `order_id`.